🧠 Proyecto: Integración Python, C y ARM64 Assembly

📌 Nombre:

Velázquez Guevara Yahir Antonio #21212061

📅 Fecha

16/04/2026

📖 Descripción

Este proyecto tiene como objetivo demostrar la integración entre lenguajes de alto nivel (Python) y bajo nivel (C y ARM64 Assembly), utilizando librerías compartidas (.so) y el módulo ctypes.

Se implementaron funciones matemáticas básicas (suma, resto, máximo) y operaciones con arreglos en Assembly, accedidas desde Python mediante un wrapper en C.

🛠️ Tecnologías utilizadas

- Python 3
- Lenguaje C
- Ensamblador ARM64
- ctypes
- Makefile
- GDB

📂 Estructura del proyecto

arm_project/ ├── asm/ │ └── math.s ├── c/ │ └── wrapper.c ├── python/ │ └── main.py ├── Makefile ├── libmath.so

⚙️ Implementación

🧩 1. Ensamblaje de Código (asm/math.s)

asm

Nombre: Velazquez Guevara Yahir Antonio Fecha: 16/04/2026 Descripción: Funciones matemáticas en ARM64 Assembly .global suma .global resta .global maximo .global suma_array

// suma(a, b) suma: suma x0, x0, x1 ret

// resta(a, b) resta: sub x0, x0, x1 ret

// maximo(a, b) maximo: cmp x0, x1 bge return_a mov x0, x1 return_a: ret

// suma de arreglo suma_array: mov x2, #0 // índice mov x3, #0 // acumulador

bucle: cmp x2, x1 bge fin

```
ldr x4, [x0, x2, LSL #3]
add x3, x3, x4

add x2, x2, #1
b loop
```

fin: mov x0, x3 ret

🧩 2. Wrapper en C (c/wrapper.c)

c Descripción: Wrapper para conectar Assembly con Python

suma larga(larga a, larga b); resto largo(largo a, largo b); largo maximo(largo a, largo b); long suma_array(long* arreglo, long n);

long suma_wrapper(long a, long b) { return suma(a, b); }

long resta_wrapper(long a, long b) { return resta(a, b); }

long maximo_wrapper(long a, long b) { return maximo(a, b); }

long suma_array_wrapper(long* arr, long n) { return suma_array(arr, n); }

🧩 3. Código Python (python/main.py)

Python Descripción: Uso de ctypes para conectar con C y Assembly

input("Presiona ENTER para continuar...")

import ctypes import numpy as np import time

lib = ctypes.CDLL("../libmath.so")

lib.suma_wrapper.argtypes = [ctypes.c_long, ctypes.c_long] lib.suma_wrapper.restype = ctypes.c_long

lib.resta_wrapper.argtypes = [ctypes.c_long, ctypes.c_long] lib.resta_wrapper.restype = ctypes.c_long

lib.maximo_wrapper.argtypes = [ctypes.c_long, ctypes.c_long] lib.maximo_wrapper.restype = ctypes.c_long

lib.suma_array_wrapper.argtypes = [ctypes.POINTER(ctypes.c_long), ctypes.c_long] lib.suma_array_wrapper.restype = ctypes.c_long

print("Suma:", lib.suma_wrapper(5, 7)) print("Resta:", lib.resta_wrapper(10, 3)) print("Max:", lib.maximo_wrapper(8, 12))

arr = np.array([1, 2, 3, 4, 5], dtype=np.int64)

resultado = lib.suma_array_wrapper(arr.ctypes.data_as(ctypes.POINTER(ctypes.c_long)), len(arr))

print("Suma array:", resultado)

Comparación de rendimiento

arr_big = np.arange(1, 1000000, dtype=np.int64)

inicio = time.time() suma_py = sum(arr_big) fin = time.time()

print("Python suma:", suma_py) print("Tiempo Python:", fin - inicio)

inicio = time.time() suma_asm = lib.suma_array_wrapper(arr_big.ctypes.data_as(ctypes.POINTER(ctypes.c_long)), len(arr_big)) fin = time.time()

print("ASM suma:", suma_asm) print("Tiempo ASM:", fin - inicio)

🧩 4. Makefile

🧩 4. Makefile

make all: gcc -g -c -fPIC asm/math.s -o math.o gcc -g -c -fPIC c/wrapper.c -o wrapper.o gcc -shared -o libmath.so math.o wrapper.o

---

▶️ Compilación

make

▶️ Ejecución

cd python python3 main.py

🐞 Uso de GDB

gdb python3 run main.py

Durante la ejecución se utilizó:

- Ctrl + C → detener ejecución
- info registers → ver registros
- bt → ver stack

⚡ Resultados

Ejemplo de salida:

Suma: 12 Resta: 7 Max: 12 Suma array: 15 Python suma: 499999500000 Tiempo Python: 0.02 ASM suma: 499999500000 Tiempo ASM: 0.005

🧠 Conclusión

El proyecto demuestra que es posible integrar Python con lenguajes de bajo nivel como C y Assembly mediante el uso de librerías compartidas.

Se observó que Assembly ofrece mejor rendimiento en operaciones repetitivas, mientras que Python facilita el desarrollo y la legibilidad del código.

El uso de GDB permitió analizar la ejecución a bajo nivel, reforzando la comprensión del funcionamiento interno del sistema.